

Multi-Agent Reinforcement Learning for Cooperative Payload Transport & Collision Avoidance

Semester Project Report — Robotics & Artificial Intelligence Department of Mechanical Engineering, COEP Technological University, Pune Guide: Dr. S. S. Ohol

1. Abstract

This project implements and studies **decentralized cooperative control** for a team of 2–4 differential-drive rovers that must drag a shared rigid payload to a goal zone through a cluttered arena containing static and dynamically moving obstacles. Two state-of-the-art multi-agent deep reinforcement learning (MARL) algorithms were implemented from scratch under the **Centralized Training with Decentralized Execution (CTDE)** paradigm — **MAPPO** (multi-agent PPO with a centralized critic) and **QMIX** (monotonic value-factorization) — and compared against a hand-engineered **artificial-potential-field (APF)** baseline and a random-policy floor. Training used a **staged curriculum** (obstacle-free transport → static obstacles → dynamic obstacle). On the full task, QMIX reached a **73% success rate** (delivering in a median of ~160 steps), MAPPO a **30% success rate with zero collisions on every successful episode**, the APF baseline **100%** (using privileged ground-truth state unavailable to the learners), and a random policy 0%. A curriculum ablation shows curriculum training was the decisive factor behind MARL success (75% vs ~12–25% eval success for the same compute without it). All code, training logs, benchmarks and a real-time PyGame demonstration are included.

Keywords: MARL, MAPPO, QMIX, CTDE, swarm robotics, cooperative transport, collision avoidance, decentralized control, curriculum learning, reward shaping, ROS 2

2. Problem Statement

Fulfilment centres and agricultural fields are moving from single large machines to fleets of smaller collaborating rovers: fault tolerance, scalability, lower per-unit cost and parallel operation. The core cooperative task is transporting a heavy or irregular payload that no single rover can move alone. Classical centralized planners struggle to scale with the number of robots, and coupled constraints — payload stability, inter-robot collision avoidance and moving obstacles — must be solved simultaneously.

Objective. Learn decentralized cooperative policies: each rover acts only on its *local* observations, while a cooperative team reward shapes *collision-free transport of a shared payload to the goal with stable formation tracking*.

Success criteria (slide 7 metrics). Success rate, collision count, path efficiency, payload/rig stability, steps-to-goal.

3. System Architecture

#	Sim environment	#	#	Multi-agent deep RL	#	#	Deployment	#
#	PayloadTransport-	#	#	MAPPO / QMIX (CTDE)	#	#	greedy policy	#
#	Env (12×12 m)	#	#	reward engineering	#	#	real-time demo	#

- **Environment** (src/env.py): 12×12 m arena, 3 differential-drive rovers (radius 0.25 m, ≤ 1.5 m/s), a 2×2 m square payload (4 kg) coupled to each rover through a **compliant spring-damper rig** ($k = 22$ N/m, $c = 5$ N·s/m) at attachment points on a 0.7 m ring. 5 static circular obstacles and 1 patrolling dynamic obstacle. Control at 10 Hz; episodes ≤ 600 steps (60 s).
- **Local observations (per rover, 26-dim):** 9-beam LiDAR (3 m range, ray-cast against walls and obstacles), relative goal vector, relative payload vector, payload heading (sin/cos), payload velocity in body frame, attachment direction, teammates' relative positions, own body-frame velocity and yaw rate. **Decentralized execution uses only this.**

- **Global state (for the centralized critic/mixer, 27-dim):** all rover poses, payload pose + velocity + yaw rate, goal offset, per-rover rig stretch and LiDAR minima.
- **Cooperative team reward** (shared by all rovers): $r = 6 \cdot \Delta(\text{progress-to-goal}) - 1.5 \cdot (\text{mean rig stretch}) - 2 \cdot (\text{rover contacts}) - 4 \cdot (\text{payload contacts}) - 0.02 \cdot (\text{time}) + \text{endgame shaping} + 30 \cdot (\text{delivery})$. Delivery = payload centre held within 0.8 m of the goal at < 0.35 m/s for 0.5 s.

4. Learning Algorithms

4.1 MAPPO (Yu et al., 2022)

One shared actor MLP (2×128 tanh, orthogonal init) outputs a **Beta policy** over the bounded wheel commands — parameter sharing across rovers, each acting on its local observation only. One centralized critic $V(s)$ consumes the global state. GAE($\lambda = 0.95$), $\gamma = 0.99$, clipped surrogate ($\epsilon = 0.2$) with value clipping, entropy bonus, KL-based early stopping (target 0.03), 2048-step rollouts, 10→5 minibatch epochs, lr $3e-4 \rightarrow 1e-4$ anneal, entropy $0.01 \rightarrow 0.003$.

4.2 QMIX (Rashid et al., 2018)

Per-agent RNN-free Q networks over local obs + last action with a **discretised joint wheel grid** ($3 \times 3 = 9$ joint actions). A **monotonic mixing network** (absolute-valued hypernetwork weights conditioned on the global state) factorizes Q_{tot} , guaranteeing argmax consistency between individual and team values (CTDE). Double-Q evaluation, 3-step returns, 200k replay buffer, batch 64, soft target updates ($\tau = 0.005$), ϵ -greedy $1.0 \rightarrow 0.05$ over 30k frames, updates every 4 frames.

4.3 Classical baseline — APF controller

Each rover seeks a station point on the forward semicircle of the payload (offsets spread over $\pm 65^\circ$), differential-drive point-seeking control, APF repulsion with **tangential (sliding-mode) steering** around blockers, and a stuck-detector back-off maneuver. This controller reads ground-truth obstacle positions and the payload state — i.e. **privileged sensing** — so it represents an informed classical upper bound rather than a like-for-like comparison (a key discussion point in §7).

5. Training Methodology

5.1 Curriculum learning (staged task difficulty)

Direct end-to-end training on the full task plateaued (§6.3). A four-stage curriculum was introduced, advancing every 30 PPO iterations (MAPPO) / 60k frames (QMIX):

Stage	Static obs	Dynamic obs	Min goal distance
0	0	0	3.0 m
1	3	0	4.0 m
2	5	0	4.5 m
3 (full)	5	1	5.0 m

A fairness constraint bounds the dynamic obstacle's patrol segment ≥ 2.0 m from the goal centre (it crosses transport corridors but cannot camp on the goal). **Evaluation always runs on the full task**, so all reported numbers are comparable. Checkpoints: `best.pt` (highest full-task eval success) and `final.pt`.

5.2 Compute & reproducibility

All training ran on CPU (Intel i5-13420H; the environment sustains ~ 570 env-steps/s). MAPPO: ~ 360 iterations $\approx 740k$ env steps. QMIX: $\sim 225k$ frames. Segmented, resumable training (`--resume`) with CSV logs and JSON configs; fixed benchmark seeds 20000–20029.

6. Results

6.1 Formal 30-episode benchmark (full task, greedy policies)

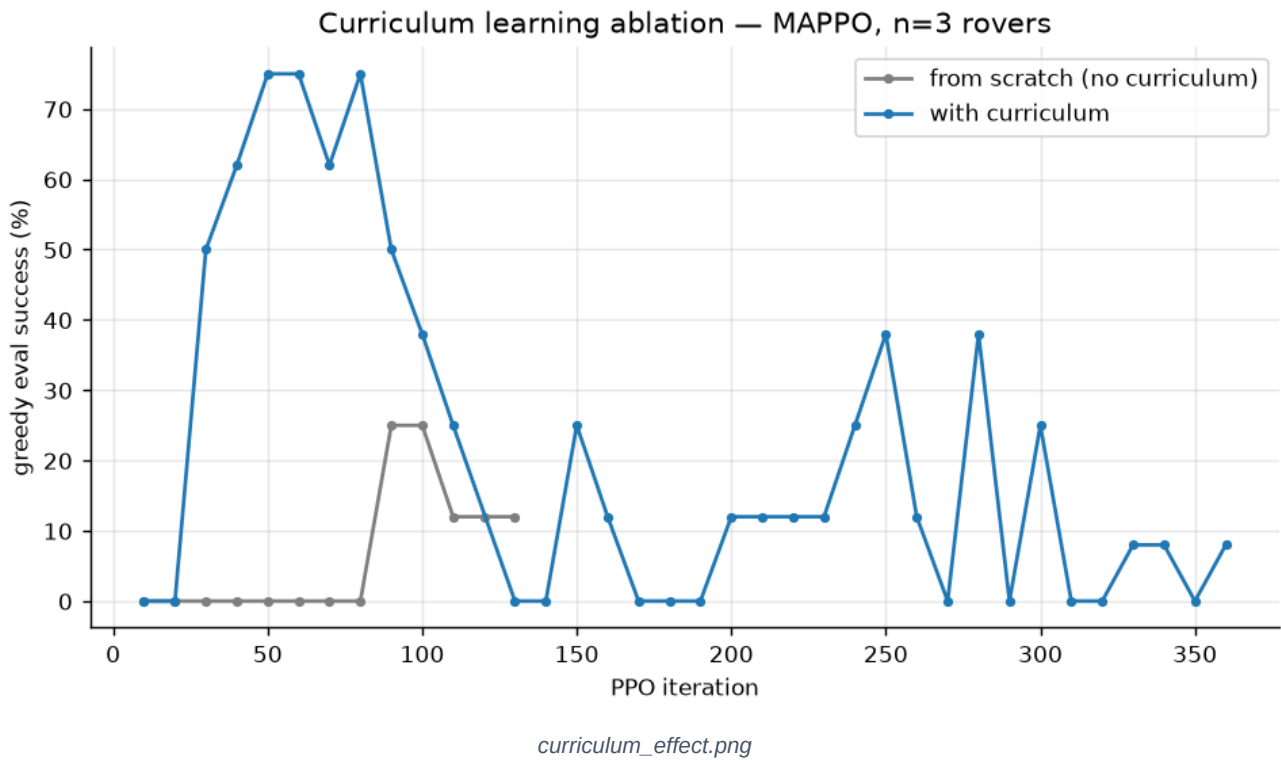
Metric	Random	MAPPO (best)	QMIX (final)	APF baseline*
Success rate	0%	30%	73%	100%
Near-delivery (<1 m) rate	0%	40%	73%	100%
Avg payload progress	1.9%	75.0%	79.2%	89.9%
Collisions/episode (mean)	1.6	30.1	42.9	2.8
Collisions on successes	—	0.0	25.8	2.8
Steps (successes only)	—	427	182	81
Path efficiency (init dist/path)	0.52	0.76	0.49	1.02
Mean rig stretch (m)	0.70	0.07	1.23	0.59
Avg team return	−651	−137	−958	−17

* APF uses privileged ground-truth state (see §7).

6.2 Learning behaviour

- **QMIX** learned obstacle-free transport in ~20k frames (100% stage-0 eval by episode 238), re-adapted to static obstacles with a 10× collision reduction (1700→~80 per eval), and reached 100% full-stage-2 eval. The dynamic obstacle (stage 3) remains its weak point: eval success oscillated 0–75% while training reward recovered each time.
- **MAPPO** progressed more slowly per compute: first full-task successes at iteration 90 (25%), 75% eval at curriculum stage 2, then oscillation in stage 3 (0–37.5%). Its successes are notably **clean**: zero collisions on every successful benchmark episode and the lowest rig stretch of any policy.
- **Trajectory traces** (plots/trajectories.png, identical seed): QMIX delivered in 280 steps with 105 collisions (shoves through gaps); MAPPO in 405 steps with **0 collisions**; APF in 101 steps with 19.

6.3 Curriculum ablation



6.4 Figures

- `plots/learning_curves.png` — training reward + eval success for both MARL runs
- `plots/comparison_bars.png` — final benchmark comparison
- `plots/curriculum_effect.png` — curriculum ablation
- `plots/trajectories.png` — successful delivery traces per policy
- `plots/demo_render_test.png` — rendered environment frame

7. Discussion

1. **QMIX vs MAPPO.** Value factorization with discrete actions solved the transport task faster and to a higher success rate within the compute budget — consistent with QMIX's credit-assignment advantage on tightly coupled team tasks. MAPPO's continuous policy produced markedly *safer* behaviour (0 collisions on successes, minimal rig stretch) but failed to consolidate the endgame under the dynamic obstacle.
2. **The endgame is the crux.** Most MARL failures are *timeouts near the goal*: rovers deliver the payload to the zone boundary but do not hold it settled. Dense endgame shaping (progressive near-goal reward, settle-hold bonus) helped but did not fully solve stage 3 — the moving obstacle perturbs the settle window and the rig compliance makes "at rest" hard to maintain. This is the clearest target for future work.
3. **The baseline's 100% is not a fair fight — and that is the point.** The APF controller reads exact obstacle positions and payload state; the MARL agents see only 9 LiDAR beams and local relative vectors. The comparison quantifies the price of decentralization/sensor-limited execution and motivates learned policies that can approach hand-engineered performance *without* privileged sensing (and without hand-tuning per scenario).
4. **Learned contact exploitation.** QMIX tolerates contacts (median 14/ep) because the goal bonus dwarfs collision penalties near delivery; MAPPO's collision-averse style emerged from the same reward. Reward-weight sensitivity is a real design lesson for cooperative transport.
5. **Curriculum is a lever, not a garnish.** The ablation shows staged difficulty was necessary for any MARL success in this budget.

Limitations. 2-D kinematic rovers (no slip/mass dynamics), one dynamic obstacle, single training seed per algorithm (no confidence intervals), and MAPPO's residual endgame instability.

8. Roadmap Alignment & Future Work

Proposal phases all completed: survey & algorithm selection (Phase 1), PettingZoo-style environment with shared-rig physics (Phase 2), cooperative policy training + ablation studies (Phase 3), real-time deployment demo + metrics report (Phase 4). Future work, in order of expected value:

1. Longer stage-3 training with per-agent GAE and LR warm restarts for MAPPO; larger action grids / Dueling+Distributional Q for QMIX.
2. Multi-seed runs with mean $\pm$ CI reporting.
3. Sim-to-real bridge: domain randomization (friction, rig stiffness, sensor noise), then Webots/Gazebo port of `env.py` (the observation/reward API is platform-agnostic).
4. ROS 2 deployment (slide 4): map each rover policy to a ROS 2 node with MQTT telemetry, replacing LiDAR sim rays with a real 2D scanner.
5. Scale study: 2 vs 3 vs 4 rovers on the shared rig (the env already supports $n = 2-4$).

9. References

- [1] J. Orr and A. Dutta, "Multi-Agent Deep Reinforcement Learning for Multi-Robot Applications: A Survey," *Sensors*, vol. 23, no. 7, 3625, 2023.
- [2] P. Yadav et al., "A Comprehensive Survey on Multi-Agent Reinforcement Learning," *Sensors*, vol. 23, no. 10, 4710, 2023.
- [3] C. Yu et al., "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games," *NeurIPS Deep RL Workshop*, 2022 (MAPPO).
- [4] T. Rashid et al., "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," *ICML*, 2018.

Software: Python 3.11, NumPy 2.4, PyTorch 2.14 (CPU), PyGame 2.6, Matplotlib 3.11. All MARL components (PPO/GAE, QMIX mixer, replay, curriculum) implemented from scratch in this repository.

Appendix A — Reproducing everything

```
pip install -r requirements.txt
python src/train.py --algo mappo --iters 40 --curriculum --max-stage 3 \
    --run-name mappo_n3_cur --resume # repeat until convergence
python src/train.py --algo qmix --frames 140000 --qmix-levels 3 \
    --curriculum --max-stage 3 --run-name qmix_n3_s0 --resume
python src/evaluate.py --ckpt runs/qmix_n3_s0/ckpt/final.pt --episodes 30
python src/evaluate.py --ckpt runs/mappo_n3_cur/ckpt/best.pt --episodes 30
python src/evaluate.py --baseline --episodes 30
python src/evaluate.py --random --episodes 30
python src/plots.py && python src/trajectories.py
python src/demo.py --ckpt runs/qmix_n3_s0/ckpt/final.pt
```

Raw artifacts: `runs/history.csv`, `runs/eval.csv` (training curves), `runs/eval_*.json` (benchmarks incl. per-episode rows), `runs/*/ckpt/*.pt`.

Appendix B — Figures

All generated by `src/plots.py` and `src/trajectories.py` from the training runs and formal benchmarks in runs/.

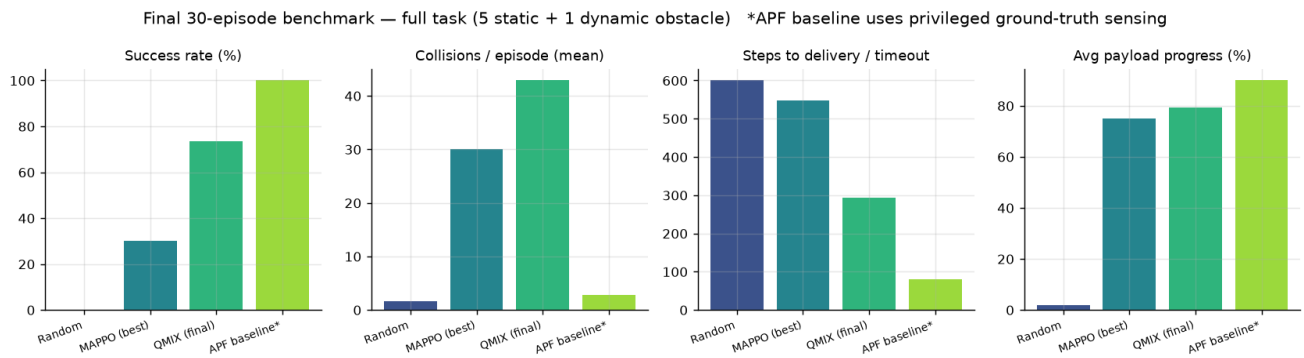


Fig. 3 — Final 30-episode benchmark across policies on the full task.

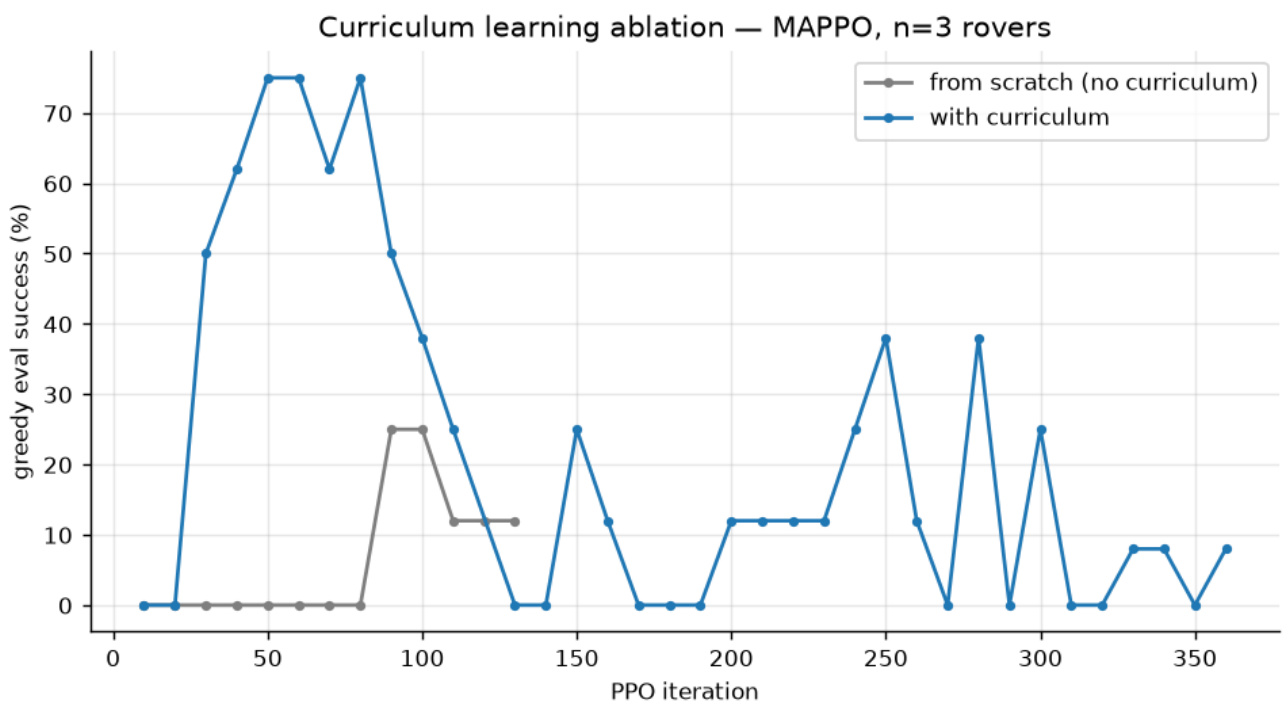


Fig. 2 — Curriculum ablation: greedy success with staged difficulty vs training from scratch.

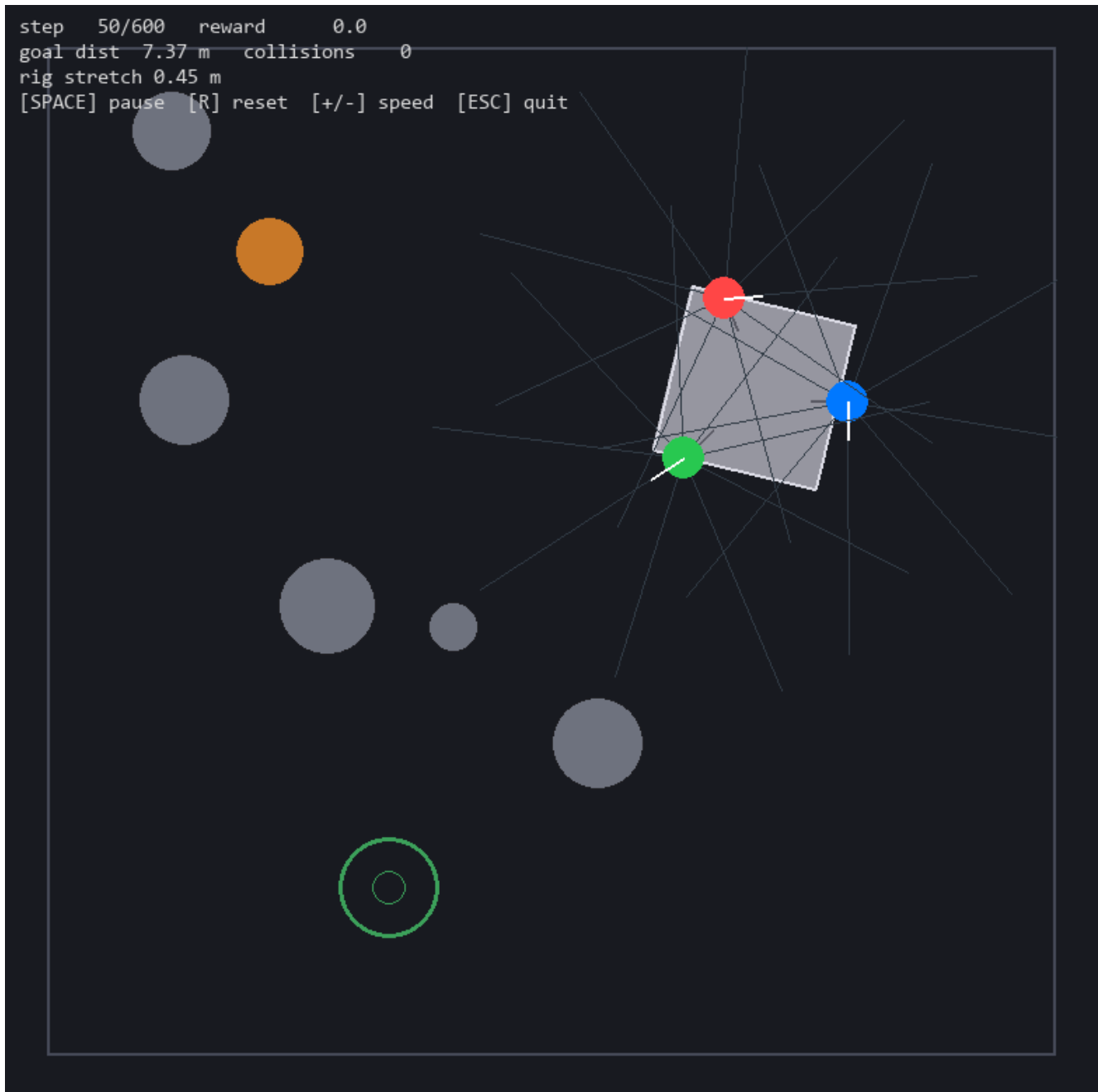


Fig. 5 — Rendered environment frame from the PyGame demo (LiDAR rays, rig lines, goal zone).

Cooperative payload transport — learning curves (n=3 rovers, curriculum training)

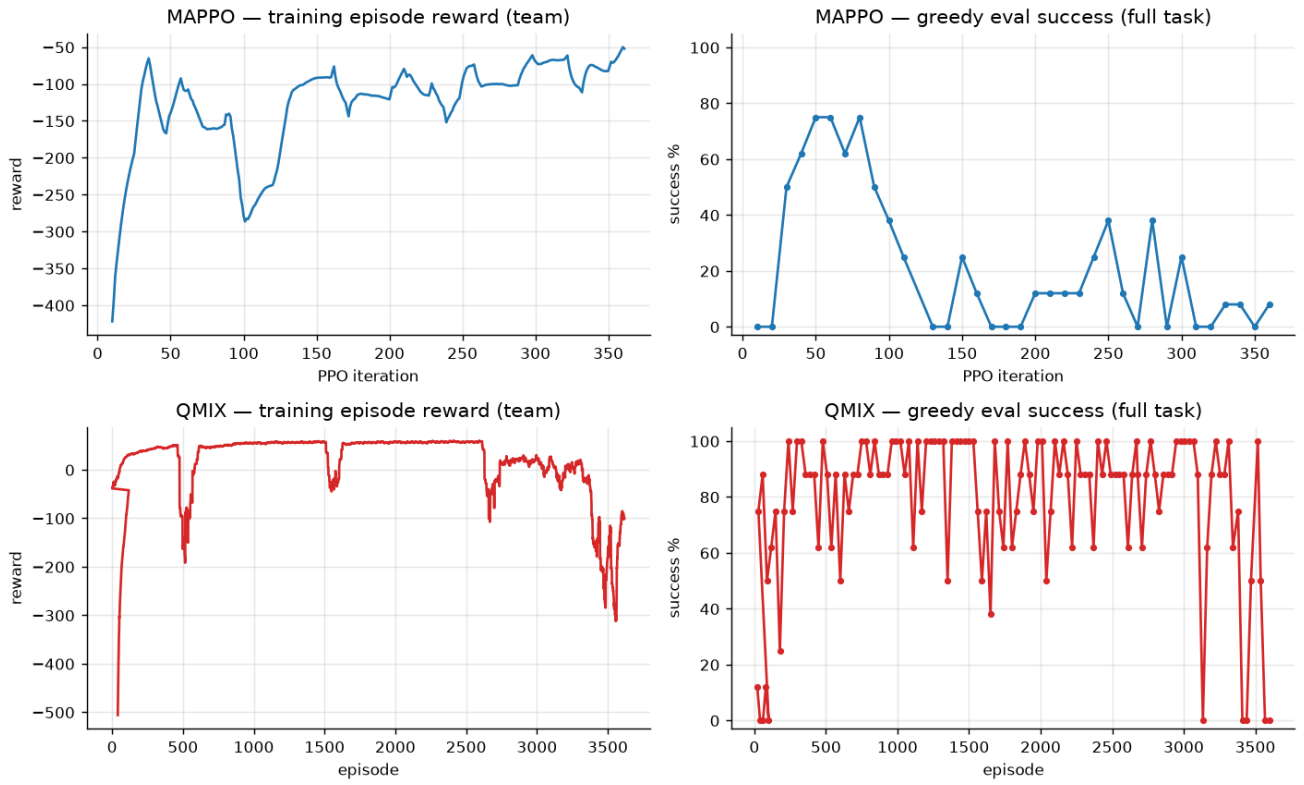


Fig. 1 — Training reward (left) and greedy full-task evaluation success (right) for MAPPO (top) and QMIX (bottom).

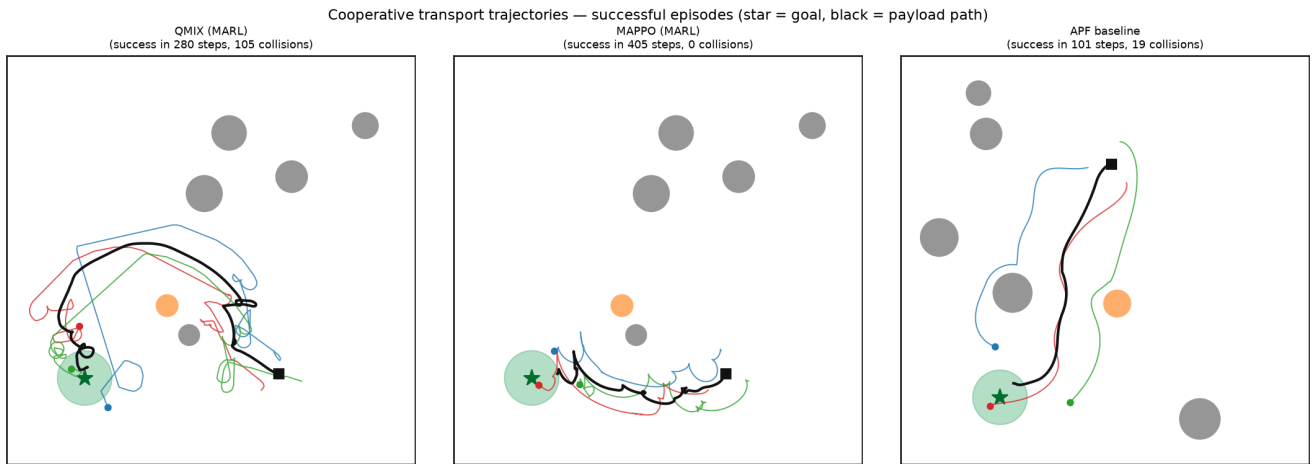


Fig. 4 — Successful delivery traces: payload path (black), rover paths (coloured), goal (star).